Excellence in Learning Innovation

LAPORAN PRAKTIKUM PROGRES 1
IMPLEMENTASI ARENA ALLOCATOR
UNTUK STRUKTUR DATA BERBASIS SISTEM

Disusun untuk

Memenuhi Tugas Mata kuliah struktur data yang diampu oleh

Bapak Muhammad Jauharul Fuady, S.T., M.T.

Disusun Oleh:

Dasteghip Fathimah Ibrahim	(250535618728)
Dendi Putra Khaeril Anam	(250535601462)
Dzauq Bachrul ‘Ulum	(250535621835)

PROGRAM STUDI S1 TEKNIK INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS NEGERI MALANG

2026

1. PENDAHULUAN

1.1 Latar Belakang

Pada pengembangan struktur data berbasis sistem, pengelolaan memori menjadi faktor penting karena memengaruhi performa dan kestabilan program. Alokasi memori dinamis yang umum digunakan memang fleksibel, tetapi pada beberapa skenario dapat menimbulkan overhead dan kompleksitas pengelolaan.

Arena Allocator dipilih karena menggunakan pendekatan alokasi linear: memori dialokasikan sekali di awal dalam satu blok besar, kemudian permintaan alokasi kecil dilayani dengan pergeseran offset. Model ini sederhana, cepat, dan sesuai untuk kebutuhan alokasi sementara yang dapat dibersihkan sekaligus melalui operasi reset.

1.2 Tujuan Penugasan

Berdasarkan deskripsi penugasan yang diberikan, tujuan dari proyek ini adalah:

1. Mengimplementasikan *Arena Allocator* sebagai *Abstract Data Type* (ADT) yang mandiri tanpa bergantung pada alokasi memori dinamis di level struktur data.
2. Membangun struktur data (seperti *array* dan *linked list*) yang seluruh elemennya berada di dalam satu blok memori terpusat.
3. Melakukan analisis komparatif antara penggunaan *Arena Allocator* dengan mekanisme malloc/free dari sisi performa dan efisiensi memori.
4. Memvisualisasikan tata letak memori dalam bentuk grid ASCII untuk memahami perilaku alokasi secara riil.

1.3 Lingkup Tahapan Minggu ke-1

Pada tahap awal ini (Minggu 1), fokus utama tim adalah membangun fondasi sistem berupa *Arena Allocator*. Implementasi difokuskan pada pembuatan fungsi-fungsi dasar API arena yang meliputi:

1) arena_init / arena_create: Inisialisasi blok memori awal dan kapasitas arena.
2) arena_alloc: Mekanisme pembagian memori di dalam arena secara linear.
3) arena_get: Pengambilan data berdasarkan *offset* yang sudah dialokasikan.
4) arena_reset: Pengosongan arena secara instan untuk memulai alokasi dari awal kembali.

1.4 Dasar Teori Singkat

Arena Allocator bekerja dengan prinsip bump allocation, yaitu menaikkan pointer atau offset setiap kali alokasi dilakukan tanpa mekanisme pembebasan per blok. Keunggulan pendekatan ini adalah biaya alokasi yang rendah dan implementasi yang mudah dipahami. Konsekuensinya, pelepasan memori umumnya dilakukan secara kolektif melalui reset seluruh arena.

2. IMPLEMENTASI PROGRAM

2.1 Konsep Implementasi

Pada tahap implementasi program ini, kami menerapkan konsep Arena Allocator sebagai blok memori utama untuk menampung alokasi data secara linear. Tahap awal difokuskan pada pembentukan arena sebagai wadah memori terpusat, kemudian program akan membagi memori tersebut melalui mekanisme offset sesuai kebutuhan alokasi data.

Sebelum masuk ke penerapan struktur data lanjutan pada progress berikutnya, kami menyiapkan fungsi-fungsi krusial untuk membangun arena, yaitu:

1. *arena_init* : Inisialisasi blok memori awal sebagai arena utama.
2. *arena_alloc* : Mekanisme pembagian memori di dalam arena secara berurutan.
3. arena_get : Pengambilan data berdasarkan *offset* hasil alokasi.
4. *Arena_reset* : Pengosongan arena secara instan untuk memulai alokasi dari awal kembali.
5. arena_destroy : Pelepasan memori arena setelah program selesai digunakan.

2.2 File arena.h

Berisikan deklarasi struktur data dan fungsi-fungsi utama yang digunakan oleh sistem arena. Struktur *Arena* didefinisikan dengan tiga komponen inti, yaitu *buffer* bertipe *unsigned char* * sebagai lokasi penyimpanan data mentah, *capacity* bertipe *size_t* sebagai ukuran total arena, dan *offset* bertipe *size_t* sebagai penanda posisi alokasi saat ini.

Pada header juga dideklarasikan fungsi-fungsi utama API arena, yaitu *arena_init*, *arena_alloc*, *arena_get*, *arena_reset*, *arena_destroy*, *arena_available*, dan *arena_dump*. Deklarasi ini bertujuan memisahkan

antarmuka (interface) dari implementasi sehingga kode lebih terstruktur dan mudah dikembangkan.

2.3 File arena.c

File ini berisi implementasi seluruh fungsi yang telah dideklarasikan pada header. Setiap fungsi memiliki peran spesifik dalam mekanisme pengelolaan memori arena.

Fungsi *arena_init* bertugas melakukan inisialisasi arena dengan mengalokasikan satu blok memori dari sistem operasi, kemudian mengatur nilai *capacity* dan *offset* awal. Apabila input tidak valid atau alokasi memori gagal, fungsi mengembalikan status gagal agar program utama dapat menangani kondisi tersebut.

Fungsi arena_alloc merupakan inti mekanisme bump allocation. Fungsi ini memeriksa apakah *offset + size* masih berada dalam batas *capacity*. Jika tidak mencukupi, fungsi mengembalikan status gagal. Jika mencukupi, nilai offset saat ini disimpan sebagai hasil alokasi, lalu offset digeser maju sebesar ukuran yang diminta.

Fungsi *arena_get* digunakan untuk menerjemahkan nilai offset menjadi alamat memori aktual. Validasi dilakukan agar offset tidak melebihi batas arena. Dengan demikian, proses pemesanan ruang (*arena_alloc*) dipisahkan dari proses pengambilan alamat data (*arena_get*) sehingga akses memori lebih aman dan terkontrol.

Fungsi *arena_reset* mengatur ulang *offset* ke nilai nol sehingga seluruh area arena dianggap kosong kembali secara logis. Proses ini tidak menghapus isi byte satu per satu, tetapi menandai bahwa alokasi baru akan dimulai dari awal.

Fungsi *arena_destroy* bertugas membebaskan blok memori arena pada akhir siklus program. Setelah pembebasan, pointer diatur ke *NULL* dan metadata arena dikembalikan ke nilai aman untuk mencegah dangling pointer.

Fungsi *arena_available* menghitung sisa kapasitas yang belum terpakai dengan rumus *capacity - offset*. Informasi ini berguna untuk pemantauan kondisi memori saat debugging maupun pengujian.

Fungsi *arena_dump* menampilkan kondisi arena secara visual dalam bentuk grid ASCII. Byte yang telah terpakai ditandai dengan *[#]* dan byte kosong ditandai dengan *[.]*. Parameter lebar tampilan digunakan agar visualisasi lebih mudah dibaca.

2.3 File main.c

File *main.c* berfungsi sebagai program pengujian implementasi arena allocator. Program dimulai dengan deklarasi variabel arena, dilanjutkan dengan inisialisasi kapasitas 32 byte, dan penampilan kondisi awal arena melalui *arena_dump*.

Setelah itu, program melakukan alokasi bertahap untuk menguji skenario normal dan skenario gagal. Alokasi blok A dan blok B digunakan untuk memverifikasi bahwa offset bergerak sesuai dengan ukuran alokasi. Data kemudian ditulis ke area memori hasil alokasi melalui pointer yang diperoleh dari *arena_get*.

Program juga menguji kasus batas dengan mencoba mengalokasikan blok C yang melebihi sisa kapasitas. Kondisi ini harus menghasilkan status gagal, sehingga dapat dibuktikan bahwa validasi batas memori berjalan dengan benar. Pada tahap akhir, *arena_reset* dipanggil untuk memastikan arena kembali ke kondisi awal secara logis.

2.5 Struktur Berkas dan Alur Uji Program

Implementasi dilakukan pada tiga berkas sumber utama, yaitu:

2.5.1 arena.h

```
#ifndef ARENA_H
#define ARENA_H

#include <stddef.h>

typedef struct {
    unsigned char *buffer;
    size_t capacity;
    size_t offset;
} Arena;

int arena_init(Arena *arena, size_t size);
int arena_alloc(Arena *arena, size_t size, size_t *out_offset);
void *arena_get(Arena *arena, size_t offset);
void arena_reset(Arena *arena);
void arena_destroy(Arena *arena);
size_t arena_available(const Arena *arena);
void arena_dump(const Arena *arena, size_t width);

#endif
```

2.5.2 arena.c

```
#include "arena.h"

#include <stdio.h>
#include <stdlib.h>

int arena_init(Arena *arena, size_t size) {
    if (arena == NULL || size == 0) {
        return 0;
    }
```

```
    arena->buffer = (unsigned char *)malloc(size);
    if (arena->buffer == NULL) {
        arena->capacity = 0;
        arena->offset = 0;
        return 0;
    }

    arena->capacity = size;
    arena->offset = 0;
    return 1;
}

int arena_alloc(Arena *arena, size_t size, size_t *out_offset) {
    if (arena == NULL || out_offset == NULL || size == 0) {
        return 0;
    }

    if (arena->offset + size > arena->capacity) {
        return 0;
    }

    *out_offset = arena->offset;
    arena->offset += size;
    return 1;
}

void *arena_get(Arena *arena, size_t offset) {
    if (arena == NULL || arena->buffer == NULL || offset >= arena-
>capacity) {
        return NULL;
    }

    return (void *)(arena->buffer + offset);
}

void arena_reset(Arena *arena) {
    if (arena == NULL) {
        return;
    }

    arena->offset = 0;
}
```

```c
void arena_destroy(Arena *arena) {
    if (arena == NULL) {
        return;
    }

    free(arena->buffer);
    arena->buffer = NULL;
    arena->capacity = 0;
    arena->offset = 0;
}

size_t arena_available(const Arena *arena) {
    if (arena == NULL || arena->capacity < arena->offset) {
        return 0;
    }

    return arena->capacity - arena->offset;
}

void arena_dump(const Arena *arena, size_t width) {
    size_t i;

    if (arena == NULL || width == 0) {
        return;
    }

    printf("\n=== ARENA DUMP ===\n");
    for (i = 0; i < arena->capacity; i++) {
        if (i % width == 0) {
            printf("\n");
        }

        if (i < arena->offset) {
            printf("[#]");
        } else {
            printf("[.]");
        }
    }

    printf("\n\nOffset: %zu / %zu\n", arena->offset, arena->capacity);
    printf("Tersisa: %zu byte\n", arena_available(arena));
}
```

2.5.3 main.c

```c
#include <stdio.h>

#include "arena.h"

int main(void) {
    Arena arena = {0};
    size_t offset_a = 0;
    size_t offset_b = 0;
    size_t offset_c = 0;
    int ok;
    size_t i;

    ok = arena_init(&arena, 32);
    if (!ok) {
        printf("Gagal inisialisasi arena.\n");
        return 1;
    }

    printf("Kondisi awal arena:\n");
    arena_dump(&arena, 8);

    ok = arena_alloc(&arena, 6, &offset_a);
    if (ok) {
        unsigned char *data_a = (unsigned char *)arena_get(&arena, offset_a);
        for (i = 0; i < 6; i++) {
            data_a[i] = 'A';
        }
        printf("\nAlokasi blok A (6 byte) berhasil di offset %zu.\n", offset_a);
    }

    ok = arena_alloc(&arena, 10, &offset_b);
    if (ok) {
        unsigned char *data_b = (unsigned char *)arena_get(&arena, offset_b);
        for (i = 0; i < 10; i++) {
            data_b[i] = 'B';
        }
        printf("Alokasi blok B (10 byte) berhasil di offset %zu.\n", offset_b);
    }
```

```
    ok = arena_alloc(&arena, 20, &offset_c);
    if (ok) {
        printf("Alokasi blok C (20 byte) berhasil di offset %zu.\n", offset_c);
    } else {
        printf("Alokasi blok C (20 byte) gagal karena kapasitas tidak cukup.\n");
    }

    printf("\nKondisi arena setelah alokasi:\n");
    arena_dump(&arena, 8);

    arena_reset(&arena);
    printf("\nSetelah arena_reset():\n");
    arena_dump(&arena, 8);

    arena_destroy(&arena);
    return 0;
}
```

2.6 Alur Uji Program

Skenario uji pada program utama disusun sebagai berikut:

1. Inisialisasi arena dengan kapasitas 32 byte.
2. Alokasi blok A sebesar 6 byte dan isi dengan karakter A.
3. Alokasi blok B sebesar 10 byte dan isi dengan karakter B.
4. Coba alokasi blok C sebesar 20 byte untuk menguji kondisi gagal.
5. Tampilkan kondisi arena sebelum dan sesudah proses reset.

3. OUTPUT PROGRAM

3.1. Ringkasan Hasil Eksekusi

Output program menunjukkan hal-hal berikut:

- Kondisi awal arena (semua slot kosong).
- Keberhasilan alokasi blok A dan B beserta offset.
- Kegagalan alokasi blok C karena melebihi kapasitas.
- Kondisi arena setelah alokasi (slot terpakai bertambah).
- Kondisi arena setelah *arena_reset()* (offset kembali 0).

```
Kondisi awal arena:

=== ARENA DUMP ===

[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 0 / 32
Tersisa: 32 byte

Alokasi blok A (6 byte) berhasil di offset 0.
Alokasi blok B (10 byte) berhasil di offset 6.
Alokasi blok C (20 byte) gagal karena kapasitas tidak cukup.


Kondisi arena setelah alokasi:

=== ARENA DUMP ===

[#][#][#][#][#][#][#][#]
[#][#][#][#][#][#][#][#]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 16 / 32
Tersisa: 16 byte

Setelah arena_reset():

=== ARENA DUMP ===

[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]
[.][.][.][.][.][.][.][.]

Offset: 0 / 32
Tersisa: 32 byte
```

4. PEMBAHASAN DAN KESIMPULAN

4.1 Pembahasan

Berdasarkan hasil eksekusi program, implementasi Arena Allocator pada Progress 1 telah berjalan sesuai rancangan awal. Kondisi arena pada tahap awal berada pada keadaan kosong, kemudian berubah menjadi terisi setelah proses alokasi blok A dan blok B. Hal ini menunjukkan bahwa mekanisme alokasi linear berbasis offset berhasil diterapkan secara konsisten.

Ketika dilakukan percobaan alokasi blok C dengan ukuran yang melebihi sisa kapasitas, sistem menolak alokasi tersebut. Perilaku ini menegaskan bahwa validasi kapasitas pada fungsi *arena_alloc* sudah bekerja dengan benar dan mampu mencegah akses memori di luar batas arena. Dari sisi keandalan, mekanisme ini penting untuk menjaga stabilitas program.

Selain itu, fungsi *arena_reset* berhasil mengembalikan offset ke posisi awal tanpa perlu membebaskan setiap blok secara individual. Pendekatan ini menjadi keunggulan utama Arena Allocator karena proses pembersihan memori logis dapat dilakukan dengan cepat dan sederhana.

Namun, hasil pembahasan juga menunjukkan keterbatasan pada tahap ini. Arena belum mendukung deallocation per-blok dan belum memiliki strategi reuse sebagian area memori. Artinya, model ini efektif untuk pola alokasi sementara, tetapi belum cukup untuk kasus yang membutuhkan pelepasan memori granular.

4.2 File `arena.h`

File header ini berisi deklarasi struktur data dan fungsi-fungsi yang akan digunakan. Struktur `Arena` didefinisikan dengan tiga komponen utama, yaitu `buffer` yang bertipe `unsigned char*` dan berfungsi sebagai tempat penyimpanan data mentah, `capacity` yang bertipe `size_t` dan menyimpan ukuran total arena, serta `offset` yang juga

bertipe `size_t` dan berfungsi sebagai penanda posisi kursor alokasi saat ini. Penggunaan tipe `unsigned char*` pada `buffer` dipilih karena ukurannya dijamin satu byte, sehingga memudahkan operasi aritmetika pointer per byte.

Pada header ini juga dideklarasikan lima fungsi utama, yaitu `arena_init` untuk inisialisasi, `arena_alloc` untuk alokasi, `arena_get` untuk mengambil pointer dari offset, `arena_reset` untuk mengosongkan arena, dan `arena_dump` untuk menampilkan kondisi arena secara visual.

4.1.2 File `arena.c`

File ini berisi implementasi dari fungsi-fungsi yang telah dideklarasikan pada header. Fungsi `arena_init` bertugas menginisialisasi arena dengan meminta blok memori ke sistem operasi melalui fungsi `malloc` sebesar parameter yang diberikan. Setelah blok memori diperoleh, kapasitas arena diatur sesuai ukuran yang diminta, dan offset diatur ke nol sebagai posisi awal. Perlu dicatat bahwa setelah inisialisasi ini, semua alokasi berikutnya tidak lagi berinteraksi dengan sistem operasi, melainkan hanya memanipulasi blok memori yang sudah diperoleh.

Fungsi `arena_alloc` merupakan inti dari mekanisme bump allocator. Fungsi ini pertama-tama memeriksa apakah jumlah offset saat ini ditambah ukuran yang diminta melebihi kapasitas arena. Jika melebihi, fungsi akan mengembalikan nilai -1 sebagai penanda kegagalan alokasi. Jika masih mencukupi, nilai offset saat ini disimpan ke dalam variabel `allocated_offset`, kemudian offset digeser maju sebesar ukuran yang dialokasikan. Nilai `allocated_offset` tersebut kemudian dikembalikan sebagai hasil. Perlu ditekankan bahwa yang dikembalikan adalah nilai offset berupa integer, bukan pointer. Pendekatan ini dipilih

karena offset bersifat stabil dan tidak akan menjadi tidak valid meskipun buffer dipindahkan.

Fungsi `arena_get` berperan sebagai penerjemah dari nilai offset menjadi pointer memori yang sesungguhnya. Fungsi ini melakukan validasi terhadap nilai offset, yaitu memastikan bahwa offset tidak negatif dan tidak melebihi kapasitas arena. Jika valid, fungsi mengembalikan pointer hasil penjumlahan alamat buffer dengan nilai offset. Inilah alasan mengapa proses alokasi dan pengaksesan data dipisahkan menjadi dua langkah, yaitu `arena_alloc` memesan ruang dan `arena_get` memberikan alamat nyata saat data akan digunakan.

Fungsi `arena_reset` cukup sederhana, yaitu hanya mengatur ulang nilai offset menjadi nol. Dengan demikian, seluruh area dalam arena dianggap kosong dan siap digunakan kembali. Data lama secara fisik masih ada dalam buffer, namun akan tertimpa pada alokasi berikutnya. Proses reset ini sangat murah secara komputasi karena tidak melibatkan pembebasan memori secara aktual.

Fungsi `arena_dump` merupakan fungsi bantu untuk debugging yang menampilkan kondisi arena secara visual. Fungsi ini melakukan iterasi sepanjang kapasitas arena dan mencetak simbol `[#]` untuk byte yang telah terpakai dan `[.]` untuk byte yang masih kosong. Parameter `width` digunakan untuk mengatur berapa banyak byte yang ditampilkan per baris agar output mudah dibaca.

4.1.3 File `main.c`

File ini berfungsi sebagai program utama untuk menguji implementasi arena allocator. Program diawali dengan deklarasi variabel `arena` bertipe `Arena`, kemudian diinisialisasi dengan kapasitas 32 byte. Kondisi awal arena ditampilkan menggunakan `arena_dump` dengan lebar 8 byte per baris, sehingga akan terlihat 32 tanda `[.]` yang menandakan seluruh arena masih kosong.

Selanjutnya, program melakukan dua alokasi berturut-turut, yaitu sebesar 5 byte yang hasil offsetnya disimpan pada variabel `a`, dan 10 byte yang hasil offsetnya disimpan pada variabel `b`. Setelah alokasi ini, offset arena akan berada pada posisi 15, dengan variabel `a` bernilai 0 dan variabel `b` bernilai 5. Kondisi arena kemudian ditampilkan kembali untuk memperlihatkan perubahannya.

Program kemudian mengisi data pada area yang telah dialokasikan. Untuk area pertama, fungsi `arena_get` dipanggil dengan offset `a`, dan hasilnya dikonversi menjadi pointer `char*` . Melalui pointer tersebut, lima byte pertama diisi dengan karakter 'A'. Proses serupa dilakukan untuk area kedua, yaitu sepuluh byte berikutnya diisi dengan karakter 'B'. Terakhir, program memanggil `arena_reset` untuk mengosongkan arena, lalu menampilkan kondisi akhir yang memperlihatkan bahwa seluruh arena kembali seperti kondisi awal.

4.2 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa pendekatan Arena Allocator berhasil mendemonstrasikan konsep manajemen memori berbasis region secara jelas. Seluruh alokasi dilakukan pada satu blok memori terpusat dengan mekanisme pergeseran offset, sehingga proses alokasi menjadi sederhana, cepat, dan mudah dipantau melalui visualisasi arena.

Implementasi pada Progress 1 juga membuktikan bahwa pemisahan antara konsep pemesanan ruang (*arena_alloc*) dan pengambilan alamat aktual (*arena_get*) memberikan stabilitas referensi yang baik selama arena masih valid. Selain itu, operasi *arena_reset* menunjukkan efisiensi dalam mendaur ulang arena karena pengosongan logis dapat dilakukan tanpa pembebasan blok satu per satu ke sistem operasi.

Di sisi lain, keterbatasan utama tahap ini adalah belum adanya deallocation per-blok dan belum tersedianya mekanisme reuse parsial untuk pola alokasi yang kompleks. Meskipun demikian, tujuan Progress 1 telah tercapai dengan baik: fondasi API arena sudah terbentuk, perilaku alokasi sudah tervalidasi, dan sistem siap dikembangkan ke Progress 2 untuk integrasi struktur data Stack berbasis arena.